DYNAMIC PROGRAMMING (DP) — PATTERNS CHEAT SHEET

Java version • Clues • Easy / Medium / Hard problems • Code templates • Start-to-end choice tree

1. DP CLUES — HOW TO IDENTIFY DP

Signal	What it means
✓ Overlapping subproblems	Same subproblem/state is solved multiple times.
✓ Optimal substructure	Optimal answer can be built from optimal smaller answers.
✓ Choices at each step	At every step you have multiple meaningful choices.
✓ Best / Max / Min / Count / Existence	Look for optimization, counting, or boolean feasibility.
✓ Can break into smaller parts	The problem naturally decomposes into smaller states.
✓ Recursion gives TLE	Naive recursive solution repeats work → memoization/DP may help.
✓ State can be defined	A small set of parameters can represent everything relevant from the past.
✓ Base cases exist	Starting conditions can be clearly defined.

2. GENERAL JAVA DP TEMPLATE

```
// Top-down memoization
long solve(State state) {
if (isBaseCase(state)) return baseValue;
if (memo.containsKey(state)) return memo.get(state);

long ans = initialValue;
for (Choice choice : choices(state)) {
long candidate = solve(nextState(state, choice));
ans = combine(ans, candidate); // min / max / sum / OR
}
memo.put(state, ans);
return ans;
}

// Bottom-up tabulation
long[][] dp = new long[n + 1][m + 1];
initializeBaseCases(dp);
for (int i = 1; i <= n; i++) {
for (int j = 1; j <= m; j++) {
dp[i][j] = transition(dp, i, j);
}
}
return dp[n][m];
```

3. THE GOLDEN DP THINKING LOOP

1. Understand the problem → What exactly is being asked?
2. Identify choices → What can I choose at this state?
3. Define state → What past information changes my future?
4. Check overlap → Can different histories reach the same state?
5. Write transition → State + choice → next state.
6. Define base case → What happens at the smallest state?
7. Choose objective → min / max / count / boolean / probability.
8. Memoize or tabulate → Avoid solving the same state repeatedly.

9. Optimize space/time → Only after the recurrence is correct.

DP PATTERNS — 1 TO 2

1. 1D DP (Linear DP)

State: dp[i] = best answer for first i elements / up to i
Clues:
• Problem depends on previous element(s)
• Optimize a sequence
• Take / skip / maximize / minimize
• Fibonacci-like recurrence

Problems

Easy: Climbing Stairs (70) • Min Cost Climbing Stairs (746)
Medium: House Robber (198) • Delete and Earn (740)
Hard: Best Time to Buy and Sell Stock IV (188) • Russian Doll Envelopes (354)

Java Template

```
long[] dp = new long[n + 1];
dp[0] = base0;
dp[1] = base1;
for (int i = 2; i <= n; i++) {
dp[i] = /* transition from dp[i-1], dp[i-2], ... */;
}
return dp[n];
```

2. 2D DP (Grid DP)

State: dp[i][j] = best answer when at cell (i,j)
Clues:
• Grid / matrix
• Move right/down or neighboring cells
• Paths, min/max sum, reachability

Problems

Easy: Unique Paths (62) • Minimum Path Sum (64)
Medium: Unique Paths II (63) • Triangle (120)
Hard: Cherry Pickup II (1463) • Dungeon Game (174)

Java Template

```
long[][] dp = new long[m][n];
dp[0][0] = base;
for (int i = 0; i < m; i++) {
for (int j = 0; j < n; j++) {
if (i == 0 && j == 0) continue;
dp[i][j] = /* best of valid previous cells */;
}
}
return dp[m - 1][n - 1];
```

DP PATTERNS — 3 TO 4

3. 0/1 Knapsack DP

State: dp[i][w] = best using first i items with capacity w

Clues:

- Items have weight/value
- Choose or do not choose
- Capacity / weight limit
- Each item used at most once

Problems

Easy: Ones and Zeroes (474)

Medium: 0/1 Knapsack (classic) • Target Sum (494)

Hard: Partition Equal Subset Sum (416) • Profitable Schemes (879)

Java Template

```
int[][] dp = new int[n + 1][capacity + 1];
for (int i = 1; i <= n; i++) {
for (int w = 0; w <= capacity; w++) {
dp[i][w] = dp[i - 1][w];
if (weight[i - 1] <= w) {
dp[i][w] = Math.max(dp[i][w],
dp[i - 1][w - weight[i - 1]] + value[i - 1]);
}
}
}
return dp[n][capacity];
```

4. Unbounded Knapsack

State: dp[w] = best answer for capacity w; item can repeat

Clues:

- Unlimited copies
- Similar to knapsack
- Take item again after choosing it
- Min/max/count with capacity

Problems

Easy: Coin Change (322)

Medium: Coin Change II (518) • Combination Sum IV (377)

Hard: Minimum Cost for Tickets (983) • Rod Cutting (classic)

Java Template

```
int[] dp = new int[amount + 1];
Arrays.fill(dp, INF);
dp[0] = 0;
for (int a = 1; a <= amount; a++) {
for (int coin : coins) {
if (coin <= a) {
dp[a] = Math.min(dp[a], dp[a - coin] + 1);
}
}
}
return dp[amount];
```

DP PATTERNS — 5 TO 6

5. Subset / Partition DP

State: dp[i][s] = whether / best using first i items to form sum s
Clues:
• Subset sum / equal partition
• Partition into groups
• Often converts to knapsack
• Boolean, count, min/max objective

Problems

Easy: Subset Sum Equal To K (416)
Medium: Count Subsets With Given Sum (494) • Target Sum (494)
Hard: Minimum Difference of Two Subsets (2035) • Partition Array (410)

Java Template

```
boolean[][] dp = new boolean[n + 1][target + 1];
dp[0][0] = true;
for (int i = 1; i <= n; i++) {
for (int s = 0; s <= target; s++) {
dp[i][s] = dp[i - 1][s];
if (s >= nums[i - 1]) {
dp[i][s] |= dp[i - 1][s - nums[i - 1]];
}
}
}
return dp[n][target];
```

6. Longest Common / Increasing Subsequence DP

State: dp[i][j] for two strings, or dp[i] for LIS
Clues:
• Two strings / sequences
• Longest / shortest subsequence
• Take / skip
• Order matters; not necessarily contiguous

Problems

Easy: LIS (300) • Longest Common Subsequence (easy variants)
Medium: LCS (1143) • Longest Increasing Subsequence (300)
Hard: Maximum Sum Increasing Subsequence • Distinct Subsequences (115) • Russian Doll Envelopes (354)

Java Template

```
// LCS
int[][] dp = new int[n + 1][m + 1];
for (int i = 1; i <= n; i++) {
for (int j = 1; j <= m; j++) {
if (a.charAt(i - 1) == b.charAt(j - 1))
dp[i][j] = dp[i - 1][j - 1] + 1;
else
dp[i][j] = Math.max(dp[i - 1][j], dp[i][j - 1]);
}
}
return dp[n][m];
```

DP PATTERNS — 7 TO 8

7. Interval DP

State: dp[l][r] = best answer for interval [l,r]
Clues:
• Range / subarray / substring
• Split at k
• Merge / remove / burst
• Choose endpoints or partition an interval

Problems

Easy: Stone Game (877) • Burst Balloons (312, basic understanding)
Medium: Palindrome Partitioning II (132) • Matrix Chain Multiplication
Hard: Burst Balloons (312) • Strange Printer (664) • Merge Stones

Java Template

```
long[][] dp = new long[n][n];
for (int len = 2; len <= n; len++) {
for (int l = 0; l + len - 1 < n; l++) {
int r = l + len - 1;
dp[l][r] = INF;
for (int k = l; k < r; k++) {
dp[l][r] = Math.min(dp[l][r],
dp[l][k] + dp[k + 1][r] + cost(l, k, r));
}
}
}
return dp[0][n - 1];
```

8. Tree DP

State: dp[node] or dp[node][state] = answer for subtree rooted at node
Clues:
• Tree / subtree
• Combine results from children
• Take / not take node
• Postorder DFS is common

Problems

Easy: Maximum Depth of Binary Tree (104) • Diameter (543)
Medium: House Robber III (337) • Binary Tree Maximum Path Sum (124)
Hard: Tree Isomorphism • Maximum Independent Set in Tree

Java Template

```
int[] dfs(TreeNode node) {
if (node == null) return new int[]{0, 0};
int[] left = dfs(node.left);
int[] right = dfs(node.right);

int take = node.val + left[1] + right[1];
int skip = Math.max(left[0], left[1])
+ Math.max(right[0], right[1]);
return new int[]{skip, take};
}
```

DP PATTERNS — 9 TO 10

9. Bitmask DP

State: dp[mask] or dp[mask][i] = answer for subset represented by mask

Clues:

- Small N (roughly <= 20)
- Visit all / choose subset / permutation
- TSP / assignment
- State compression

Problems

Easy: Counting Bits (338) • Can I Win (464)

Medium: Minimum Cost to Visit All Nodes (847)

Hard: Traveling Salesman Problem • Hamiltonian Path • Assignment Problem

Java Template

```
int N = 1 << n;
long[][] dp = new long[N][n];
for (long[] row : dp) Arrays.fill(row, INF);
dp[1][0] = 0; // start at node 0

for (int mask = 1; mask < N; mask++) {
for (int u = 0; u < n; u++) {
if ((mask & (1 << u)) == 0) continue;
for (int v = 0; v < n; v++) {
if ((mask & (1 << v)) != 0) continue;
int next = mask | (1 << v);
dp[next][v] = Math.min(dp[next][v],
dp[mask][u] + cost[u][v]);
}
}
}
return Arrays.stream(dp[N - 1]).min().orElse(INF);
```

10. Digit DP

State: dp[pos][tight][started][state] = answer while constructing digits

Clues:

- Numbers in range [L,R]
- Count numbers satisfying digit property
- N can be very large (up to 10^18)
- Digit restrictions / digit sum

Problems

Easy: Count Numbers With Unique Digits (357)

Medium: Numbers With Repeated Digits (1012)

Hard: Count Special Integers • Non-negative Integers Without Consecutive Ones (600)

Java Template

```
long solve(long x) {
// Typical state:
// dp[pos][tight][started][otherState]
// tight: prefix is equal to x
// started: non-leading-zero digit has appeared
return digitDp(x);
}
```

4. DP CHOICE TREE — FROM START TO END

START	Understand what the problem asks
↓	
Q1	Do I have meaningful choices at each step?
NO	Consider Greedy / Math / Graph / Simulation / other approach
YES	Continue
Q2	Can the problem be broken into smaller subproblems?
NO	Probably not DP
YES	Continue
Q3	Do different paths repeat the same subproblem/state?
NO	Maybe recursion / greedy / other technique
YES	DP candidate
Q4	What structure defines the state?
1D	Linear sequence → 1D DP
2D	Grid / matrix → 2D DP
WEIGHT	Items + capacity + take/skip → Knapsack
TWO SEQUENCES	Two strings/sequences → LCS / subsequence DP
INCREASING	Increasing/decreasing subsequence → LIS DP
RANGE	Interval [l,r], split/merge/remove → Interval DP
TREE	Subtree / combine children → Tree DP
SMALL N	Subset/permutation, N≈20 → Bitmask DP
DIGITS	Count numbers / digit restrictions → Digit DP
→	Define State → Write Transition → Base Cases → Compute → Answer
END	Optimize only after correctness

QUICK PATTERN PICKER

Problem clue	Go to
Linear decisions / one-by-one	1D DP
Grid / matrix / paths	2D Grid DP
Items + weight/value + capacity	0/1 or Unbounded Knapsack
Subset / equal sum / partition	Subset / Knapsack DP
Increasing subsequence	LIS DP
Two strings / common subsequence	LCS / String DP
Range / split / merge / remove	Interval DP
Tree / subtree / combine children	Tree DP
Small N / subsets / visit all	Bitmask DP
Digits / range [L,R] / digit restrictions	Digit DP

5. DP QUESTIONS — EASY → MEDIUM → HARD

EASY — Build Basics

1. Climbing Stairs (70)	2. Min Cost Climbing Stairs (746)
3. Fibonacci Number (509)	4. House Robber (198)
5. Best Time to Buy and Sell Stock	6. Palindrome Number (DP version)
7. Unique Paths (62)	8. Ransom Note (DP version)
9. Decode Ways (91)	10. N-th Tribonacci Number (1137)
11. Counting Bits (338)	12. Subset Sum Equal To K (416)

MEDIUM — Level Up

1. Longest Increasing Subsequence (300)	2. Coin Change (322)
3. 0/1 Knapsack (classic)	4. Target Sum (494)
5. Partition Equal Subset Sum (416)	6. Minimum Path Sum (64)
7. Triangle (120)	8. Edit Distance (72)
9. Maximum Product Subarray (152)	10. Burst Balloons (312 — Interval DP)
11. LCS (1143)	12. House Robber III (337)

HARD — Master DP

1. Regular Expression Matching (10)	2. Wildcard Matching (44)
3. Interleaving String (97)	4. Scramble String (87)
5. Distinct Subsequences (115)	6. Russian Doll Envelopes (354)
7. Stone Game II / IV	8. Number of Ways to Stay in the Same Place After Some Steps (1269)
9. TSP (Bitmask DP)	10. Count Numbers With Sum K (Digit DP)
11. Cherry Pickup II (1463)	12. Strange Printer (664)
13. Matrix Chain Multiplication	

6. GOLDEN RULES TO REVIEW BEFORE EVERY DP PROBLEM

1. Define the state clearly — what does dp[...] mean?
2. List every choice from that state.
3. Write the recurrence before coding.
4. Define base cases carefully.
5. Choose min / max / count / boolean / probability.
6. Check whether different histories reach the same state.
7. Memoize first if recursion is easier to reason about.
8. Convert to tabulation only after the recurrence is correct.
9. Optimize memory only after correctness.
10. Always ask: “What information from the past actually affects my future?”